

集成

使用 Kimi CLI 调用 Kimi 大模型

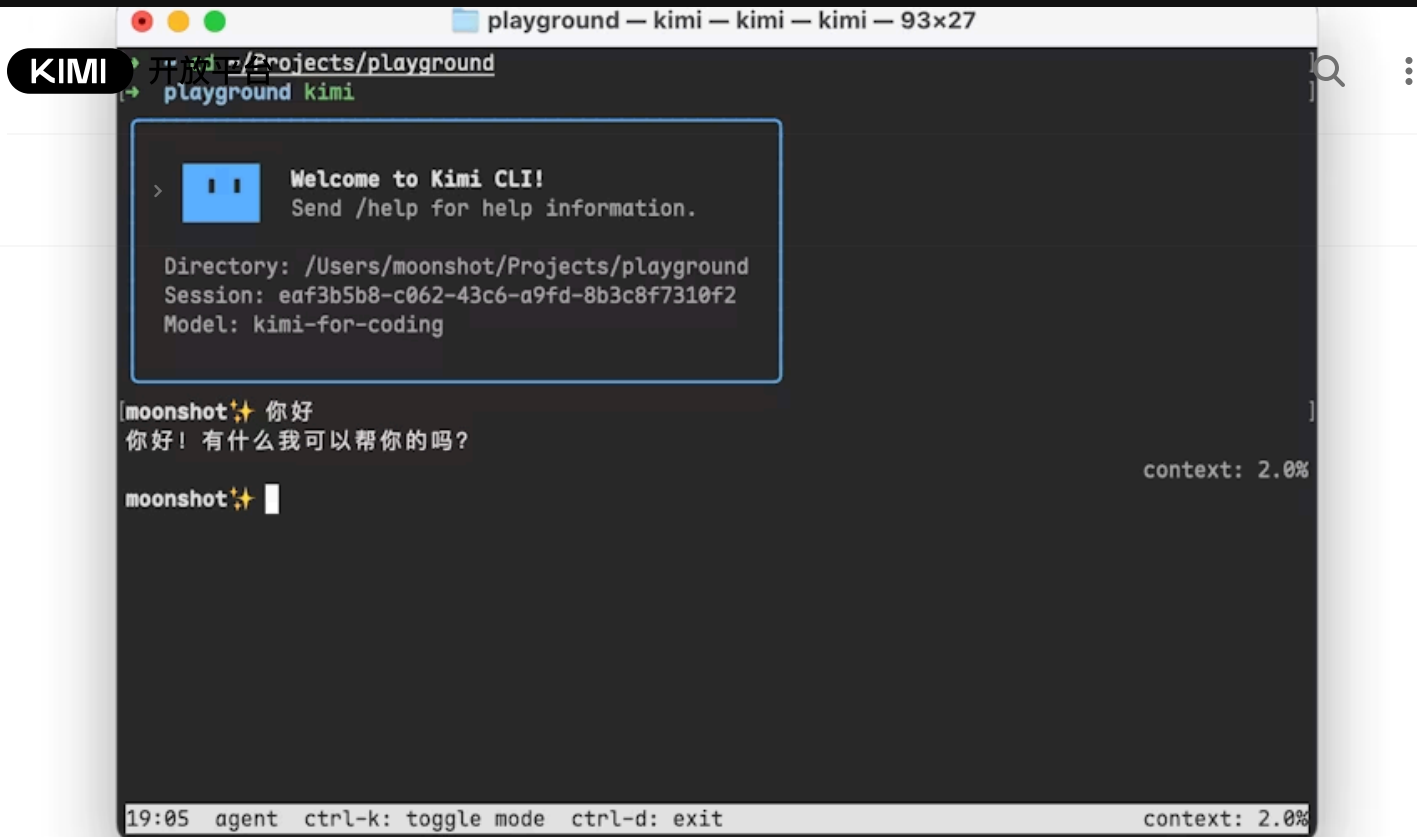
📄 复制页面



Kimi CLI 是 Moonshot AI 自研的命令行通用智能体工具，它可以帮助你快速完成各种各样的编程和文件处理等任务。

Kimi CLI 目前还在 Technical Preview 阶段，如遇到 bug 或有任何意见或建议，欢迎通过 <https://github.com/MoonshotAI/kimi-cli/issues> 提交反馈！

🚀 Kimi K2.6 模型已正式发布，长程代码编写能力更强更稳。



```
playground — kimi — kimi — 93x27
/Users/moonshot/Projects/playground
playground kimi

Welcome to Kimi CLI!
Send /help for help information.

Directory: /Users/moonshot/Projects/playground
Session: eaf3b5b8-c062-43c6-a9fd-8b3c8f7310f2
Model: kimi-for-coding

[moonshot🌟 你好
你好！有什么我可以帮你的吗?
moonshot🌟 ]
context: 2.0%

19:05 agent ctrl-k: toggle mode ctrl-d: exit context: 2.0%
```

安装

Kimi CLI 支持 macOS 和 Linux 系统，要求使用 uv 包管理器安装。

如果你的系统中还没有安装 uv，请先参考 [uv 安装说明](#) 进行安装。通常，在 macOS 和 Linux 系统中，可使用以下命令安装 uv：

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

安装 uv 后，使用以下命令安装 Kimi CLI：

```
uv tool install --python 3.13 kimi-cli
```

由于 macOS 的安全校验机制，在 macOS 上第一次运行可能需要较长时间，请耐心等待。可以尝试将你所使用的终端工具添加到「系统设置」-「隐私与安全性」-「开发者工具」中，以信任终端上运行的程序。

升级

使用以下命令升级 Kimi CLI：

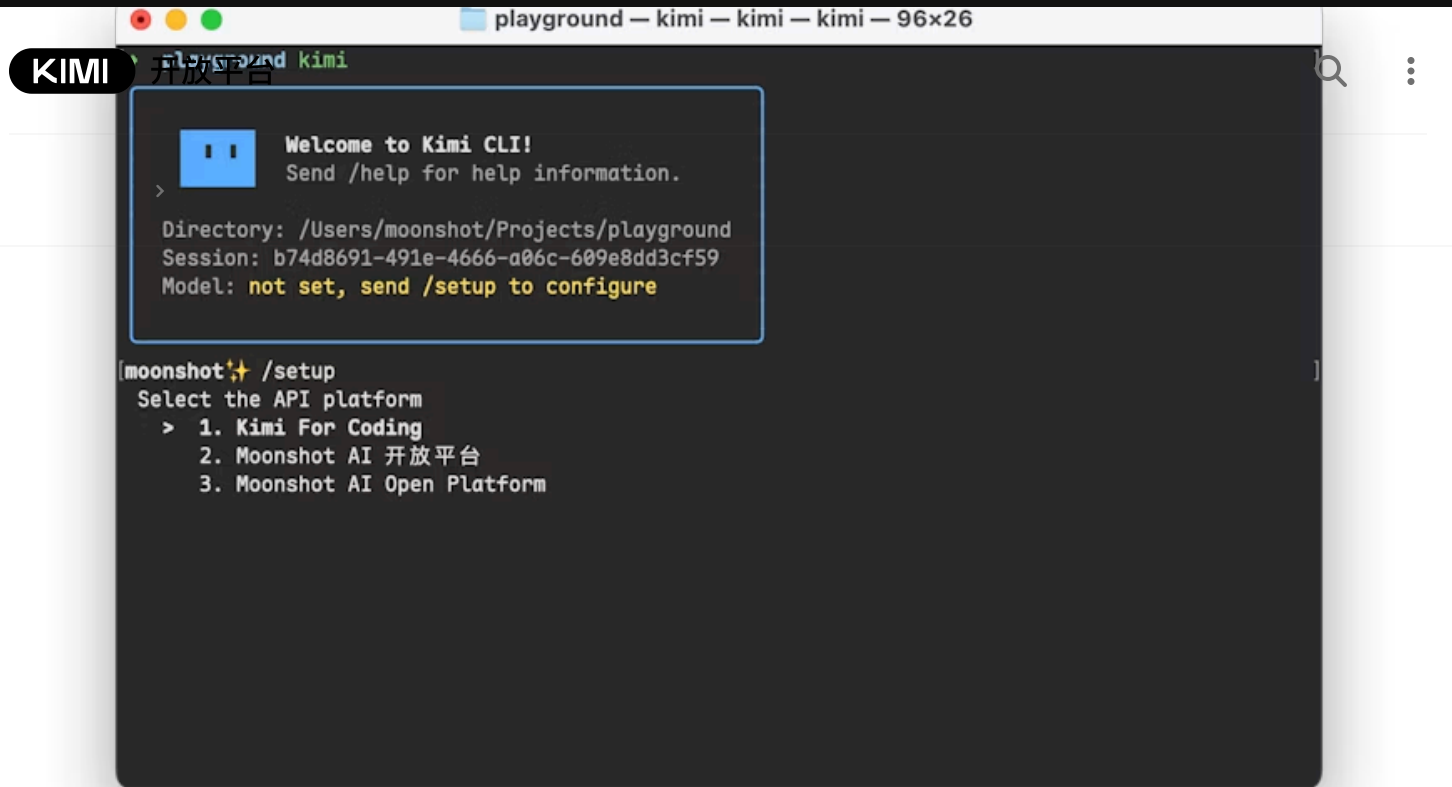
```
uv tool upgrade kimi-cli --no-cache
```

使用

在命令行中进入你想要 Kimi CLI 操作的项目目录，运行 `kimi` 命令，即可启动 Kimi CLI。例如：

```
cd my-project  
kimi
```

首次运行时，Kimi CLI 会提示没有配置模型，需输入 `/setup` 元命令，进入配置流程：



- Kimi For Coding 会员权益用户，选择第一个「Kimi For Coding」，在随后的提示中，输入在会员页面获得的 API Key，并选择 `kimi-for-coding` 模型；Moonshot AI 开放平台用户，根据提示选择对应的平台，输入 API Key 并选择想要使用的模型。[查看 Kimi For Coding 会员权益说明](#)
- 使用开放平台 API Key 的用户，请选择第二个「Moonshot AI 开放平台」，然后配置在开放平台创建的 API Key [创建 API Key](#)，请确保 API Key 有足够的余额，否则可能会出现 API 调用失败的情况。

配置完成后，即可开始使用 Kimi CLI，例如：

KIMI

开放平台



Welcome to Kimi CLI!
Send /help for help information.

>Directory: /Users/moonshot/Projects/playground
Session: a29007fe-2dfd-46a9-8565-f15ede9834ab
Model: kimi-k2-turbo-preview

[moonshot🌟 写一个 2048 小游戏

我为你创建一个2048小游戏。让我先设置任务清单，然后开始实现。

✓ Used SetTodoList:

- 创建HTML结构和基本样式 [Pending]
- 实现游戏逻辑（移动、合并、计分） [Pending]
- 添加键盘事件处理 [Pending]
- 添加游戏状态管理（开始、结束、胜利） [Pending]
- 添加重新开始功能 [Pending]

context: 2.6%

✓ Used SetTodoList:

- 创建HTML结构和基本样式 [In Progress]
- 实现游戏逻辑（移动、合并、计分） [Pending]
- 添加键盘事件处理 [Pending]
- 添加游戏状态管理（开始、结束、胜利） [Pending]
- 添加重新开始功能 [Pending]

context: 2.7%

首先创建HTML文件：

✓ Used WriteFile: index.html

context: 2.8%

✓ Used SetTodoList:

- 创建HTML结构和基本样式 [Done]
- 实现游戏逻辑（移动、合并、计分） [In Progress]
- 添加键盘事件处理 [Pending]
- 添加游戏状态管理（开始、结束、胜利） [Pending]
- 添加重新开始功能 [Pending]

context: 3.7%

现在创建JavaScript游戏逻辑文件：

✓ Used WriteFile: game.js

context: 3.8%

✓ Used SetTodoList:

- 创建HTML结构和基本样式 [Done]
- 实现游戏逻辑（移动、合并、计分） [Done]
- 添加键盘事件处理 [Done]
- 添加游戏状态管理（开始、结束、胜利） [Done]
- 添加重新开始功能 [Done]

context: 5.2%

完成！我已经为你创建了一个完整的2048小游戏。游戏包含以下功能：

主要特性：

1. ****完整的游戏逻辑**** - 支持四个方向（上下左右）的移动和方块合并
2. ****计分系统**** - 实时显示当前得分和历史最高分
3. ****键盘控制**** - 使用方向键控制游戏
4. ****触摸支持**** - 支持移动设备的触摸滑动操作
5. ****游戏状态管理**** - 胜利和失败的提示
6. ****重新开始**** - 随时可以重新开始游戏
7. ****数据持久化**** - 最高分保存在本地存储中

Kimi CLI 不仅仅是一个编程智能体，还可以通过 Ctrl-K 快捷键切换到 shell 模式。通过该模式，你：
KIMI 开放平台 可以在不离开 Kimi CLI 的情况下，直接执行 shell 命令，方便进行文件操作和查看结果。例如：

```
>

playground — kimi — kimi — kimi • kimi — 95x39

Welcome to Kimi CLI!
Send /help for help information.

Directory: /Users/moonshot/Projects/playground
Session: f79d4cf5-0482-40ef-880a-0725fc3a07f8
Model: kimi-k2-turbo-preview

[moonshot$ ls
game.js      index.html
[moonshot$ head -n 10 game.js
class Game2048 {
  constructor() {
    this.size = 4;
    this.grid = [];
    this.score = 0;
    this.best = parseInt(localStorage.getItem('best2048') || '0');
    this.won = false;
    this.over = false;

    this.tileContainer = document.getElementById('tile-container');
[moonshot$ git 提交一下
✖ Used Bash: git status
Failed with exit code: 128

当前目录不是一个 Git 仓库。让我先初始化 Git 仓库，然后添加并提交文件：
✔ Used Bash: git init
context: 2.6%

✔ Used Bash: git add .
context: 2.7%

✔ Used Bash: git commit -m "Initial co....js and index.html files"
context: 2.7%

Git 提交已完成！已成功：
- 初始化 Git 仓库
- 添加所有文件到暂存区
- 提交更改，提交信息为 "Initial commit: Add game.js and index.html files"
context: 2.7%

context: 2.8%
```

搭配 Zsh 使用

Zsh 用户可以搭配 [zsh-kimi-cli](#) 插件，在 shell 中快速调用 Kimi CLI。

🚀 Kimi K2.6 模型已正式发布，长程代码编写能力更强更稳。

KIMI 开放平台

clone from <https://github.com/MoonshotAI/zsh-kimi-cli.git> \

`${ZSH_CUSTOM:-~/.oh-my-zsh/custom}/plugins/kimi-cli`



然后在 `~/.zshrc` 中启用该插件：

```
plugins=(... kimi-cli)
```

重新启动 Zsh 之后，即可在 Zsh 中通过 Ctrl-K 进入 Kimi CLI 模式：

```
playground — moonshot@moonshotdeMacBook-Pro — ..ts/playground — -zsh — 93x26
[→ playground git:(main)
[→ playground git:(main) ✨ 你好
你好！有什么我可以帮你的吗？
context: 2.6%
→ playground git:(main) ✨
```

目前 zsh-kimi-cli 插件还在持续更新中，请定期前往 `custom/plugins/kimi-cli` 目录通过 `git pull` 拉取更新。

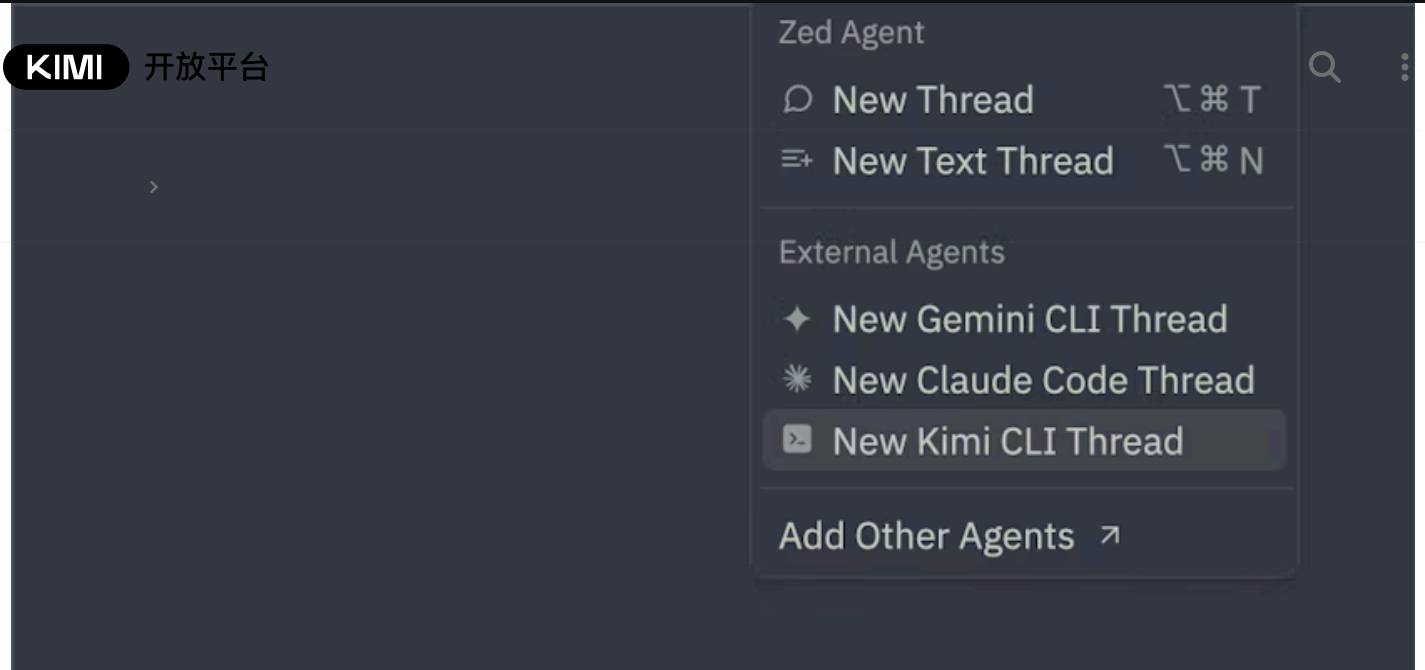
Kimi CLI 原生提供 **Agent Client Protocol** 支持，可以搭配任何 ACP 客户端使用，例如 **Zed 编辑器**。

ACP 是 Zed 编辑器推出的一种通用智能体协议，使智能体的核心功能（服务端）和用户界面（客户端）解耦，用户可以选择不同的智能体服务端和客户端进行搭配使用。

要在 Zed 中使用 Kimi CLI，首先需确保已经安装并配置好 Kimi CLI，然后在 Zed 配置文件（`~/.config/zed/settings.json`）中添加以下内容：

```
{
  "agent_servers": {
    "Kimi CLI": {
      "command": "kimi",
      "args": ["--acp"],
      "env": {}
    }
  }
}
```

随后即可在 Zed 侧边栏创建 Kimi CLI Thread：



接入 MCP 工具

Kimi CLI 支持广泛采用的 MCP 配置格式指定 MCP 工具。例如：

```
{
  "mcpServers": {
    "context7": {
      "url": "https://mcp.context7.com/mcp",
      "headers": {
        "CONTEXT7_API_KEY": "YOUR_API_KEY"
      }
    },
    "chrome-devtools": {
      "command": "npx",
      "args": ["-y", "chrome-devtools-mcp@latest"]
    }
  }
}
```

启动时，通过 `--mcp-config-file` 参数指定 MCP 配置文件路径即可。例如：

Kimi CLI 更多用法

除了上述功能，还可以通过 `kimi --help` 查看 Kimi CLI 的更多用法。或者访问 Kimi CLI 官方网站了解更多 [Kimi CLI 文档](#)。

Kimi Agent SDK

Kimi Agent SDK 提供了以编程方式与 Kimi CLI 交互的接口，支持在代码中集成 Kimi 智能体能力。通过 SDK，你可以：

- **构建自定义应用** - 将 Kimi Agent 集成到自己的工具和流程中
- **自动化任务** - 编写复杂的多轮对话脚本
- **扩展功能** - 注册自定义工具供模型调用
- **处理审批** - 程序化响应权限请求

SDK 目前支持以下语言：

- **Go** - [go](#)
- **Node.js** - [node/agent_sdk](#)
- **Python** - [python](#)

详细信息请查看 [Kimi Agent SDK GitHub 仓库](#)。

此页面对您有帮助吗？

👍 是

👎 否

🎉 Kimi K2.6 模型已正式发布，长程代码编写能力更强更稳。



KIMI

开放平台



>